

CSE 321 - Introduction to Algorithm Design

Homework 05

1. Answer questions below. a) Why do we use greedy algorithms? b) Do greedy algorithms always fail to find globally optimal solution? Why or why not?

Provide concrete examples.

① Greedy Algoritmalar, bir kısıla bağlı tüm sonuçları en geniş anlamı ile açıklamaya çalışan bir fonksiyonu, beldezen en iyi sonucu verdiği gör önünde bulundurularak sürekli çalıştırılır.

- Aralıklı çözümleri saptamak için kullanılır.
- kullanışlı oldukları için hızlı çalışırlar.
- Sıklıkla en iyi tahmini verirler.
- Genellikle çok hızlı hesaplanır ve kavramsal olarak basit oldukları için yazımı kolaydır.
- Lokal olarak en iyi çözümleri verir.

② Sonraki adımın en belirgin fayda sağlayacağına karar vererek, karmaşık, çok adımlı problemlere basit, kolay uygulanabilir çözümler arayan algoritmalar oldukları için kullanılırlar.

③ Her zaman global olarak seçilen çözüm optimal solution'u vermek zorunda değildir. Ama her zaman da yanlış çözüme ulaştırıyor diyemeyiz. Çünkü greedy algoritmalar temel olarak local'deki en iyiyi odaklanır. Algoritma oluşturulurken global etki gör önünde bulundurulmaz. Bunun için kesin bir yargıya bulunmak doğru değildir. Genel olarak 4 fonksiyondan oluştuğunu gösterelimiz.

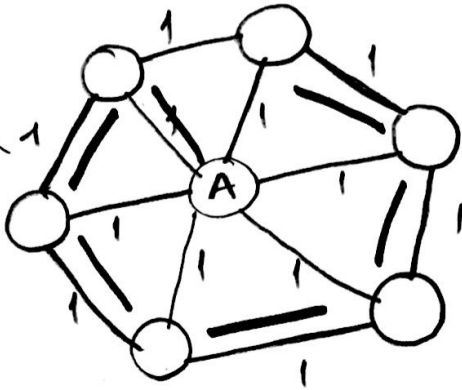
- Seçilen öğe kümesinin bir çözüm sağlandığını kontrol eden bir işlev
- kümenin olabirlihesi kontrol eden bir işlev
- Seçme işlemi
- Açıkça görülmeyen objektif, bir işlev, bir çözümün değerini verir.

2. Prove or disprove the following statements.

a) If e is a minimum-weight edge in a connected weighted graph, it must be among edges of at least one minimum spanning tree of the graph.

Aynı graph üzerinde
değerlendirirsek
Prim algoritması
kullanılarak seçilen edge'leri
tüm her türlü en kısa uzunluğa
sahip edge MST'nin içinde
bulunacaktır.
TRUE

b) If e is a minimum-weight edge in a connected weighted graph, it must be among edges of each minimum spanning tree of the graph.



1. Yal kırmızı ile gösterilmiştir.

2. Yal mavi ile gösterilmiştir.

Tüm edge'lerin ağırlığı en az
değere sahip olmasına rağmen
her spanning tree o edge'leri
kullanmak zorunda değildir.
Bundan dolayı; FALSE

c) If edge weights of a connected weighted graph are all distinct, the graph must have exactly one minimum spanning tree.

MST bulma algoritması sırasıyla;

→ Herhangi bir noktadan başlayarak komşularına bakar ve en küçük olanı seçer.

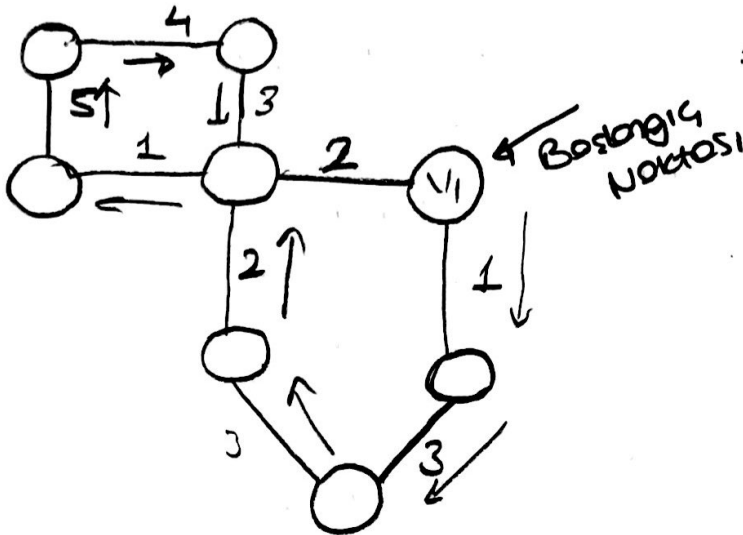
→ Aynı işlem tüm vertexlerden geçinceye kadar devam eder.

→ Her adımda daha önceden o vertexin ziyaret edilip edilmediği bilgisi de tutularak tekrar aynı vertexin seçilmesi engellenmiş olur. Beraber cycle yaparsa da o denkleme alınmaz.

→ Bu algoritma çalıştığında minimum değerleri barındıran yolların geçmesi olacağından exactly bir MST 'nin var olduğunu söyleyebiliriz.

TRUE

d) If edge weights of a connected weighted graph are not all distinct, the graph must have more than one minimum spanning tree.



→ Başlangıç noktası seçilen v_1 vertexinden itibaren aynı uzunluğa sahip edge'lerin olması rağmen tek bir MST oluşmuştur. Birden fazla oluşmasına gerek kalmamıştır.

3. Design a greedy algorithm to solve 0/1 Knapsack Problem¹. Describe your greedy approach in detail. Write a program to show your algorithm including test on a sample set. Write code in Python. Analyze its best case, worst case, and average case complexities.

0/1 Knapsack Problem: Greedy olarak çözölenememektedir. (Optimal Solution Verememektedir)
Örnekler üzerinden Ağırlarsak;

Knapsack Capacity $\rightarrow W=25$

Item	A	B	C	D
Price	12	9	9	5
Size	24	10	10	7

Optimal Greedy Approach
ilk olarak Price'yi en yüksek olanı AL

Size = 24
Price = 12 $\rightarrow$ NOT OPTIMAL

Optimal olarak $10+10 \Rightarrow$ SIZE
Orjinal $10+10 \Rightarrow$ PRICE

2) Knapsack Capacity $\rightarrow W=30$

Item	A	B	C
Price	50	140	60
Size	5	20	10
Ratio	10	7	6

Ratio'su En yüksek olan Seçilirse;

$$5+20=25$$

$$50+140=190$$

Orjinal optimal çözüm;
B ve C

$$\text{Size} = 20+10=30$$

$$\text{Price} = 140+60=200$$

4. Design a greedy algorithm to solve Travelling Salesman Problem (TSP) ² with greedy approach. Describe your greedy approach in detail. Write a program to show your algorithm including test on a sample set. Write code in Python. Analyze its best case, worst case, and average case complexities.

Kruskal algoritmasındaki genel mantık kullanılmaktadır (Sırasıyla)

→ Artan sırayla küçükten büyüğe sıralarız, (Edge'leri)

→ En az cost'a sahip edge seçilerek ilerleriz. İlerlerken seçilen edge'lerin cycle oluşturmamasına ve daha önceden seçilip seçilmeme durumu kontrol edilir.

(Non-Cycle & Non-Selected Edges)

→ Rastgele seçilen bir şehirden başlayarak oradan sonraki en kısa mesafe seçilir. Sırasıyla bu şekilde ilerler.

	Şehir1	Şehir2	Şehir3	Şehir4
Şehir1	0	10	15	6
Şehir2	10	0	9	12
Şehir3	15	9	0	8
Şehir4	6	12	8	0

↑
Graph'ı iki boyutlu
→ Array şeklinde
represent ettiğimizi
forzedelim

Random Sayının 1 olarak
atandığını forzedebiliriz. Tur;
Şehir1 → Şehir4 → Şehir3 → Şehir2

şeklinde olacaktır.

$$6 + 8 + 9 = 23$$

Burada en optimal çözümü
vermiş olmama rağmen
her zaman optimal çözüm
verdiğini söyleyemeyiz

```
def TSPGreedy(Array[ ][ ], random X,  
              CityCount)
```

```
Array[ ] selected
```

```
for i ← 0 to CityCount - 1
```

```
for j ← 0 to CityCount - 1
```

```
minVal ← Array[X][0]
```

```
if minVal > Array[X][i]
```

```
minVal = Array[X][i]
```

```
index = i
```

```
end if
```

```
end for
```

```
selected[j] = minVal
```

```
X = index
```

```
end for
```

```
for i ← 0 to CityCount - 1
```

```
print ( Şehir i → selected[i])
```

```
# Yandaki Array varsayılırsa
```

```
Şehir1 → Şehir4 → Şehir3 → Şehir2
```

Scanned by CamScanner

Algoritmanın Complexity 'si:

$$\sum_{i=0}^n \sum_{j=0}^n 1 = \sum_{i=0}^n n \\ = n^2 \in \Theta(n^2)$$

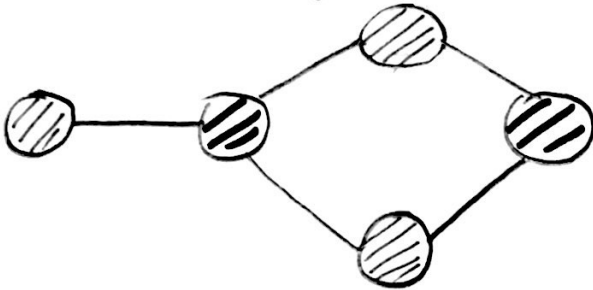
İçerdiği if koşuluna her zaman gireceği için
Best Case, Worst Case, Average Case Complexity'leri
 $\Theta(n^2)$ 'dir.

5. Design a greedy algorithm to solve Map Coloring Problem³. Describe your greedy approach in detail. Write a program to show your algorithm including test on a sample set. Write code in Python. Analyze its best case, worst case, and average case complexities.

Graph'da vertex 'ler ile ona komşu olan vertexlerin aynı renge boyanmaması amaçlanır. Bunu yaparken renklerin de kendi içleri-
de dereceleri bulunduğu için bunu da hesaba katmak gereklidir.
④ En az renk kullanarak boyanmak amaçlanır.

Algoritma;

- ① Vertexleri derecelerine göre büyükten küçüğe sırala
- ② En büyük dereceli düğümden başlanarak, kullanılmayan bir renk ile boyanır.
- ③ Komşu olan düğümlere farklı bir renk seçilerek boyama yapılır.
- ④ Komşu olmayan düğümler ise seçilen renk ile boyanır.



Map Coloring Probleminde;

→ Haritayı diktlemisel bir graph olarak görebiliriz. Grafın noktaları en az üç değişik ülkenin sınırının kesişimi, kenarları da iki ülkenin ortak sınır olsun. Yalnız, iki bölgenin sınırda olması için iki bölgenin kenarlarının birkaç noktada kesişmeleri yetmez; iki bölge ancak ortak kenarları varsa sınırda sayılırlar.

Algorithm

```
def color(map):  
    while uncolored:
```

```
        coloredwithcurrent = set()
```

```
        a = uncolored.popleft()
```

```
        colors[a] = currentcolor
```

```
        coloredwithcurrent.add(a)
```

```
        for vertex in copy(uncolored):
```

```
            if not neighborof(vertex, coloredwithcurrent, map):
```

```
                uncolored.remove(vertex)
```

```
                colors[vertex] = currentcolor
```

```
                coloredwithcurrent.add(vertex)
```

```
        currentcolor += 1
```

```
    return colors
```

Vertex Count $\rightarrow V$

Color Count $\rightarrow C$

$$\left. \begin{array}{l} \text{Vertex Count} \rightarrow V \\ \text{Color Count} \rightarrow C \end{array} \right\} \text{ is } \sum_{i=0}^C \sum_{j=0}^V 1 = C \cdot V$$

Best Case

Worst Case

Average Case

Complexity of this Algorithm is $\underline{O(CV)}$ dir.

6. Consider the problem of scheduling n jobs of known durations $t_1, \dots, t_n$ for execution by a single processor. The jobs can be executed in any order, one job at a time. You want to find a schedule that minimizes the total time spent by all the jobs in the system. (The time spent by one job in the system is the sum of the time spent by this job in waiting plus the time spent on its execution.)

Design a greedy algorithm for this problem. Does the greedy algorithm always yield an optimal solution? Prove or disprove.

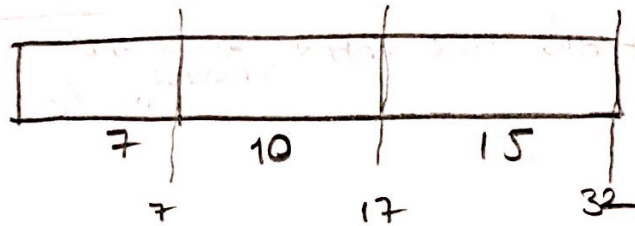
① En kısa iş ilk yapılır temelli bir algoritma kullanılabilir.
O sırada elde bulunan işleri bitirmek için gereken süreye göre sıralanır.
En kısa iş öncelik verilerek ona göre sıralanır.

İşlem CPU zamanı

A	10
B	15
C	7

Yanda verilen işler aynı anda başlatılır ve
sıra ile beklerler.

İşlem	zaman	İşlem	Çalışma	Kolon
C	0	C	7	0
en kısa olan işi	7	A	10	0
bitirerek	17	B	15	0
başlatılır				



$32/3 \approx 11$ Ortalama bekleme süreleri

Execution time'leri
sıralamak için
selection sort vb.
algoritmalar kullanılabilir.

② Gelen işlerin
sıramaları önce
gelene göre alınıp
yapılmadığı için optimum
değeri vermez.

Fakat, bu işlemler sıradan
yeni gelen instruction'ların
ne kadar bitirilmesi alınması
bu algoritmanın verimliliğini
düşürür.

7. Design a greedy algorithm for the assignment problem (see Section 3.4). Does your greedy algorithm always yield an optimal solution? Prove or disprove.

Greedy Algorithm

- ① → matrix'deki en büyük elemanı bul.
- ② → 0 elemanın bulunduğu satırı ve sütunu bul, matrix'den kaldır. 0 elemanı da yeni bir listede tut.
- ③ → 1 ve 2. adımları matrix bitene kadar tekrar et

Bu algoritma $O(n^3)$ zaman alır.

Daha iyi bir algoritma dynamic programming ile $O(n^2 \log n)$ ile çalıştırılabilir ve aynı zamanda

1	2	3
4	5	6
8	7	9

$9 + 5 + 1 = 15$ X

fakat

1	2	3
4	5	6
8	7	9

$8 + 6 + 2 = 16$

↑
Optimal Solution

Bu matrix algoritmasıyla yapıldığında en iyi çözüm vermediğini gösterilebiliriz. Optimal solution vermez

8. Write a pseudocode of the greedy algorithm for the change-making problem, with an amount n and coin denominations $d_1 > d_2 > \dots > d_m$ as its input. What is the time efficiency class of your algorithm?

Algorithm ChangeMaking($n, D[1..m]$)

for $i \leftarrow 1$ to m do

$C[i] \leftarrow \lfloor n / D[i] \rfloor$

$n \leftarrow n \bmod D[i]$

if $n = 0$ return C

else return "no solution"

Time Efficiency $\rightarrow \underline{\underline{\Theta(m)}}$

9. Answer questions below. Discuss.

a) How can we use Prim's algorithm to find a spanning tree of a connected graph with no weights on its edges?

Tüm edge'leri 1 yaparak Prim algoritmasını çalıştırabiliriz,

ya da tüm

edge'ler

aynı olarak

her yerde farklı

bir değer

b) Is it a good algorithm for this problem?

Graph üzerinde dolaşabilmek için DFS veya BFS kullanarak basitçe, sparse graflarda kolaylıkla uygulanabilir ve algoritmanın etkili kullanılması sağlanabilir.